

Lecture 11

Polymorphism, Part 3 Extensibility

Example Code and Exercises

<https://github.com/eecs498-software-design/lecture>



Liskov Substitution Principle

Examples:

- An interface T implemented by S.
- A base class T extended by S.

- If S is a **subtype** of T, an S may be substituted for a T.
- More formally, any property provable about a T should also hold for an S.
- For **method signatures** that are implemented/overridden in S:
 - Return types: Covariant from T to S.
 - Parameter types: Contravariant from T to S.
 - Exception types thrown: Covariant from T to S.
- For **behavior** of the subtype S or its methods, compared to T:
 - Do not weaken **Postconditions**
 - Do not strengthen **Preconditions**
 - Do not weaken **Invariants**
 - Maintain **The History Rule**



Liskov Substitution Principle

Examples:

- An interface T implemented by S.
- A base class T extended by S.

- If S is a **subtype** of T, an S may be substituted for a T.
- For **method signatures** that are implemented/overridden in S:
 - Return types: Covariant from T to S.
 - Parameter types: Contravariant from T to S.
 - Exception types thrown: Covariant from T to S.



Liskov Substitution Principle

Examples:

- An interface T implemented by S.
- A base class T extended by S.

- If S is a **subtype** of T, an S may be substituted for a T.
- For **behavior** of the subtype S or its methods, compared to T:
 - Do not weaken **Postconditions**
 - Do not strengthen **Preconditions**
 - Do not weaken **Invariants**
 - Maintain **The History Rule**



Related Concepts

Properties of Objects

- **Immutability** An object whose data cannot be changed.

Properties of Functions

- **Purity** No side effects and output depends only on inputs.
- **Referential Transparency** Can replace $f(x)$ with precomputed output.
- **Idempotency** Calling multiple times is the same as calling once.



The Open/Closed Principle (OCP)

- “software entities (classes, modules, functions, etc.) should be open for extension, but closed for modification”*
- i.e. Add features by adding, not modifying, **abstractions** to avoid requiring changes to other parts of the system (or breaking them!)
- This is a heuristic and must be balanced against other factors in context.
OCP is most essential in cases where interfaces are expected to be stable.
 - e.g. The codebase has significant maturity/complexity to make refactoring prohibitive.
 - e.g. Many people/teams depend on a common abstraction and volatility is prohibitive.
 - e.g. Interfaces have been “published” and breaking changes are prohibitive.

*The traditional formulation of the OCP is generally focused on object-oriented programming and favors the use of subtype polymorphism. We'll take a more general view.

The Expression Problem

- A tension between providing **extensibility of both types and operations**.

		Operations on the Types/Variants			
		calculateSize()	audit()	export()	...more?
Types/ Variants	File	code	code	code	code
	Folder	code	code	code	code
	Shortcut	code	code	code	code
	...more?	code	code	code	code

Goal: Extend in a type-safe way (i.e. catch missing cases).

Goal: Extend without modifying/recompiling existing code.



Subtype Polymorphism

```
abstract class DriveNode {  
  name: string; owner: string;  
  owner: string;  
  sharedWith: Set<string>;  
  
  constructor(name: string, owner: string) {  
    this.name = name;  
    this.owner = owner;  
    this.sharedWith = new Set();  
  }  
  
  // Each operation must be implemented by subclasses  
  abstract calculateSize(): number;  
  abstract auditPermissions(): string[];  
  abstract export(depth?: number): string;  
}
```

Add a new operation?

✗ Add a new abstract method. It's type safe, but you must modify all existing derived types.

```
class DriveFile extends DriveNode {  
  sizeBytes: number;  
  mimeType: string;  
  
  constructor(name: string, owner: string,  
              sizeBytes: number, mimeType: string) {  
    super(name, owner);  
    this.sizeBytes = sizeBytes;  
    this.mimeType = mimeType;  
  }  
  
  calculateSize(): number {  
    return this.sizeBytes;  
  }  
  
  auditPermissions(): string[] {  
    if (this.sharedWith.size > 0) {  
      return [`File '${this.name}' shared with:  
${[...this.sharedWith].join(", ")}`];  
    }  
    return [];  
  }  
  
  export(depth = 0): string {  
    return " ".repeat(depth) + `📄 ${this.name}\n`;  
  }  
}
```

Add a new type?

✓ Add a new derived class.



Ad Hoc Polymorphism

Add a new operation?

✓ Add a new function operating on the union.

```
type DriveNode = DriveFile | DriveFolder | DriveShortcut;
```

```
interface NodeBase {  
  name: string;  
  owner: string;  
  sharedWith: Set<string>;  
}
```

```
interface DriveFile extends NodeBase {  
  kind: "file";  
  sizeBytes: number;  
  mimeType: string;  
}
```

```
interface DriveFolder extends NodeBase {  
  kind: "folder";  
  children: DriveNode[];  
}
```

```
interface DriveShortcut extends NodeBase {  
  kind: "shortcut";  
  target: DriveNode;  
}
```

Add a new type?

✗ Add a new variant to the union.
It's type safe if you have an exhaustiveness check, but you must modify all the switches.

```
function calculateSize(node: DriveNode): number {  
  switch (node.kind) {  
    case "file":  
      return node.sizeBytes;  
    case "folder":  
      return node.children.reduce(  
        (sum, child) => sum + calculateSize(child), 0  
      );  
    case "shortcut":  
      return 0; // Links don't count  
    default:  
      return assertNever(node); // exhaustiveness check  
  }  
}
```

```
function auditPermissions(node: DriveNode): string[] {  
  const result: string[] = [];  
  if (node.sharedWith.size > 0) {  
    const type = node.kind.charAt(0).toUpperCase() +  
      node.kind.slice(1);  
    result.push(`${type} '${node.name}' shared with:  
      ${[...node.sharedWith].join(", ")}`);  
  }  
  if (node.kind === "folder") {  
    for (const child of node.children) {  
      result.push(...auditPermissions(child));  
    }  
  }  
  // may want to consider if an exhaustiveness  
  return result; // check is required or if this always works  
}
```



The Visitor Pattern

Ad Hoc Poly

```
class Node {
public:
    Node(const string& name, const string& owner) { ... }
    virtual ~Node() = default;
    virtual void accept(NodeVisitor& v) const = 0; // ACCEPT
    const string& name() const { ... }
    const string& owner() const { ... }
    const set<string>& sharedWith() const { ... }
    void addSharedWith(const string& user) { ... }
};
```

```
class File : public Node {
public:
    File(const string& name, const string& owner,
         size_t size, const string& mime) { ... }
    void accept(NodeVisitor& v) const override {
        v.visit(*this);
    }
    size_t sizeBytes() const { ... }
    const string& mimeType() const { ... }
};
```

```
class Folder : public Node {
public:
    Folder(const string& name, const string& owner) { ... }
    void accept(NodeVisitor& v) const override {
        v.visit(*this);
    }
    void addChild(shared_ptr<Node> child) { ... }
    const vector<shared_ptr<Node>>& children() const { ... }
};
```

Base OP Operations = Visitor

```
class NodeVisitor {
public:
    virtual ~NodeVisitor() = default;
    virtual void visit(const File& file) = 0;
    virtual void visit(const Folder& folder) = 0;
    virtual void visit(const Shortcut& shortcut) = 0;
};
```

Add a new function to a type hierarchy without modifying the classes in the hierarchy.

overloading

Specific Operation

```
class SizeCalculator : public NodeVisitor {
public:
    void visit(const File& f) override { _totalBytes += f.sizeBytes(); }
    void visit(const Folder& f) override {
        for (const auto& child : f.children()) child->accept(*this);
    }
    void visit(const Shortcut& s) override { /* links don't count */ }
    size_t totalBytes() const { return _totalBytes; }
private:
    size_t _totalBytes = 0;
};
```

Node*

this → visit(child)

old

Node* node = ...
sizeCalculator calc;

calc → visit(*node)

node → accept(calc)

```
class PermissionAuditor : public NodeVisitor {
public:
    void visit(const File& f) override { audit(f, "File"); }
    void visit(const Folder& f) override {
        audit(f, "Folder");
        for (const auto& child : f.children()) child->accept(*this);
    }
    void visit(const Shortcut& s) override { audit(s, "Shortcut"); }
    const vector<string>& report() const { return _report; }
private:
    void audit(const Node& n, const string& type) {
        if (!n.sharedWith().empty()) { ... }
    }
    vector<string> _report;
};
```

Double Dispatch

Break Time

<https://bit.ly/30Lkhvb>

